

Все, что нужно знать о математике трансформеров

Часть 4 из [Как масштабировать свою модель](#) ([Часть 3: Шардинг](#) | [Часть 5: Обучение](#))

Здесь мы кратко рассмотрим архитектуру трансформера, в частности то, как рассчитывать количество операций с плавающей запятой, байты и другие интересные нас величины.

АВТОРЫ
[Джейкоб Остин](#)
[Шолто Дуглас](#)
[Рой Фростиг](#)
[Ансельм Левская](#)
[Чарли Чен](#)
[Шарад Викрам](#)

[Федерико Леброн](#)
[Питер Чой](#)
[Винай Рамасеш](#)
[Альберт Уэбсон](#)
[Райнер Поуп*](#)

МЕСТО РАБОТЫ
Google DeepMind

ОПУБЛИКОВАНО
4 февраля 2025 года

Counting Dots

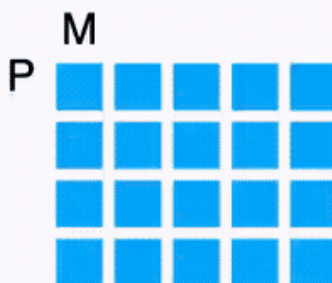
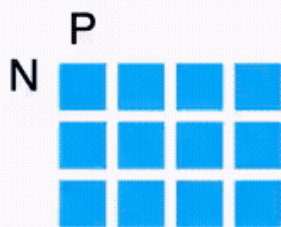
Начнем с векторов x, y и матрицы A, B следующих форм:

array	shape
x	$[P]$
y	$[P]$
A	$[N\ P]$
B	$[P\ M]$

- Точечный продукт из $x \cdot y$ требует P складывает и умножает, или $2P$ Общее количество операций с плавающей запятой.
- Произведение матрицы на вектор Ax делает N скалярные произведения вдоль строк A , для $2NP$ Флопс.
- Матричное произведение AB выполняет матрично-векторное умножение для каждого из M столбцы из B , для $2NPM$ Общее количество операций над числами с плавающей запятой.
- В общем случае, если у нас есть два массива более высокой размерности C и D , где некоторые измерения **СУЖАЮТСЯ**, а некоторые **УВЕЛИЧИВАЮТСЯ** (например, $C[GH\mathbf{IJKL}]$, $D[G\mathbf{H}MN\mathbf{KL}]$), то стоимость этого сокращения в FLOPS в два раза превышает произведение всех C и D размеры, в которых размеры пакета и сжатия учитываются только один раз (например, $2GH\mathbf{I}JMN\mathbf{KL}$). Обратите внимание, что размерность является объединяющей только в том случае, если она присутствует в обоих множителях. (Также обратите внимание, что коэффициент 2 не применяется, если нет сокращающих размерностей и выполняется только поэлементное умножение.)¹

Operation	FLOPs	Data
$x \cdot y$	$2P$	$2P$
Ax	$2NP$	$NP + P$
AB	$2NPM$	$NP + PM$
$[c_0, \dots, c_N] \cdot [d_0, \dots, d_N]$	$2 \prod c_i \times \prod_{\substack{d_i \neq \text{BATCH} \\ d_j \neq \text{CONTRACT}}} d_j$	$\prod c_i + \prod d_j$

Обратите внимание, что при матричном умножении *вычислительная мощность* масштабируется в кубе. $O(N^3)$ в то время как передача данных масштабируется только квадратично $O(N^2)$ Это означает, что по мере увеличения размера матричного умножения *становится проще* достичь предела вычислительных ресурсов. Это крайне необычно и во многом объясняет, почему мы используем архитектуры, в которых преобладает матричное умножение — их можно масштабировать!



Прямые и обратные вычисления с плавающей запятой

Во время обучения нам не так важен результат умножения матриц, как его производная. Оказывается, вычисление этой производной обходится примерно в три раза дороже, чем само умножение матриц.

Если представить, что **B** — это всего лишь одна матрица в более крупной сети, а **A** — наши входные активации с **C = A B**, то производная функции потерь **L** по **B** определяется по правилу дифференцирования сложной функции:

$$\frac{\partial L}{\partial B} = \frac{\partial L}{\partial C} \frac{\partial C}{\partial B} = A^T \left(\frac{\partial L}{\partial C} \right)$$

который требует **2NPM** Число операций с плавающей запятой в секунду для вычисления (поскольку оно сокращается за счет **N** размерность). Аналогично, производная потерь по **A** равна

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial C} \frac{\partial C}{\partial A} = \left(\frac{\partial L}{\partial C} \right) B^T$$

что опять же **2NPM** FLOPs, поскольку **dL/dC** — это матрица размера $[N, M]$ Хотя эта величина не является производной по какому-либо параметру, она используется для вычисления производных для предыдущих слоев сети (например, так же, как dL/dC используется для вычисления dL/dB, описанного выше).

Сложив эти значения, мы видим, что **во время обучения у нас в общей сложности 6NPM операций с плавающей запятой**, в то время как при логическом выводе их 2NPM: 2NPM при прямом проходе и 4NPM при обратном. Поскольку PM — это количество параметров в матрице, это простейшая форма знаменитого **6 * num parameters * num tokens** аппроксимация провалов трансформатора во время обучения: для каждого токена требуется **6 * num parameters** FLOPs. Более правильный вывод мы приведем ниже.

Трансформерный учет

Трансформеры — это будущее. Ну, по крайней мере, настоящее. Возможно, еще несколько лет назад они были одной из многих архитектур. Но сегодня стоит знать практически все детали этой архитектуры. Мы не будем заново представлять ее, но [этот блог](#) и [оригинальная статья о трансформерах](#) могут оказаться полезными.

Вот базовая схема архитектуры декодера трансформера:

Глобальный расчет FLOPS и параметров

Давайте рассчитаем количество FLOPS для каждого слоя трансформера (чтобы не приходилось везде умножать на **L**). Обратите внимание, что количество FLOPS при обучении почти всегда в три раза превышает количество FLOPS при логическом выводе, поэтому любое общее значение можно разделить на 3, чтобы узнать стоимость только прямого прохода.

Многослойные перцептроны

Многослойные перцептроны в трансформерах обычно состоят из двух входных матричных умножений, которые объединяются поэлементно, и одного выходного матричного умножения:

operation	train FLOPs	params
$A[B, T, D] \cdot W_{in1}[D, F]$	$6BTDF$	DF
$A[B, T, D] \cdot W_{in2}[D, F]$	$6BTDF$	DF
$\sigma(A_{in1}[B, T, F] * A_{in2}[B, T, F])$	$O(BTF)$	
$A[B, T, F] \cdot W_{out}[F, D]$	$6BTDF$	DF
	$\approx 18BTDF$	$3DF$

Attention

Для общего случая внимания к сгруппированным запросам с разным количеством головок **Q** и **KV** предположим, что размерность головки **H** одинакова для проекций **Q, K, V**, и оценим стоимость матричных умножений **QKV**:

operation	train FLOPs	params
$A[B, T, D] \cdot W_Q[D, N, H]$	$6BTDNH$	DNH
$A[B, T, D] \cdot W_K[D, K, H]$	$6BTDKH$	DKH
$A[B, T, D] \cdot W_V[D, K, H]$	$6BTDKH$	DKH
$A[B, T, N, H] \cdot W_O[N, H, D]$	$6BTDNH$	DNH
	$12BTD(N + K)H$	$2D(N + K)H$

Операция внимания в виде скалярного произведения более тонкая и по сути является **TH · HS** matmul с пакетной обработкой **B, K** размеры, функция softmax и **TS · SH** matmul снова применяется пакетно **B, K** размеры. Мы выделяем сгруппированные затемнения синим цветом:

operation	train FLOPs
$Q[B, T, K, G, H] \cdot K[B, S, K, H]$	$6BTSKGH = 6BTSNH$
$\text{softmax}_S L[B, T, S, K, G]$	$O(BTSKG) = O(BTSN)$
$S[B, T, S, K, G] \cdot V[B, S, K, H]$	$6BTSKGH = 6BTSNH$
	$\approx 12BTSNH = 12BT^2NH$

Примечание [причинно-следственная маскировка]: в большинстве современных трансформеров используется причинно-следственная маскировка, а не полное двунаправленное внимание. В этом случае количество операций с плавающей запятой, необходимых для вычисления скалярного произведения, уменьшается вдвое. Чтобы добиться такого снижения на практике, нам нужно использовать ядро внимания, а не простое матричное умножение.

Другие операции

В трансформере происходит несколько других операций. Слои нормализации сравнительно дешевы, и при оценке затрат первого порядка ими можно пренебречь. Обратите внимание, что в каждом слое их обычно две (одна перед механизмом внимания, другая — перед многослойным перцептроном). Кроме того, есть еще одно огромное (хотя и не для каждого слоя) матричное умножение при извлечении.

operation	train FLOPs	params
$2 \times \text{layernorm}_D \ A[B, T, D]$	$O(BTD)$	$2D$
$A[B, T, D] \cdot W_{unembed}[D, V]$	$6BTDV$	DV

General rule of thumb for Transformer FLOPs

If we neglect the cost of dot-product attention (which is reasonable for shorter-context training), then the total FLOPs across all layers is

$$(18BTD F + 12BTD(N + K)H)L = 6 * BT * (3DF + 2D(N + K)H)L \\ = 6 * \text{num tokens} * \text{parameter count}$$

This leads to a famous rule of thumb for estimating dense Transformer FLOP count, ignoring the attention FLOPs. (Unembedding is another simple matmul with $6BTDV$ FLOPs and DV params, and follows the same rule of thumb.)

Fractional cost of attention with context length

If we do account for dot-product attention above and assume $F = 4D$, $D = NH$ (as is typical) and $N = K$, the ratio of dot-product attention FLOPs to all matmul FLOPs (including the attention projections) is:

$$\frac{\text{attention FLOPs}}{\text{matmul FLOPs}} = \frac{12BT^2NH}{18BTD F + 24BTDNH} = \frac{12BT^2D}{4 * 18BTD^2 + 24BTD^2} = \frac{12BT^2D}{96BTD^2} = \frac{T}{8D}$$

The upshot is that **dot-product attention FLOPs only become dominant during training once $T > 8D$** . For $D \sim 8k$, this would be $\sim 64k$ tokens. This makes some sense, since it means as the MLP size increases, the attention FLOPs become less critical. For large models, the quadratic cost of attention is not actually a huge obstacle to longer-context training. However, for smaller models, e.g. Gemma-27B with $D=4608$, attention becomes dominant around 37k sequence lengths.² Flash Attention also helps alleviate the cost of long-context, which we discuss briefly [in Appendix A](#).

Miscellaneous Math

Sparsity and Mixture-of-Experts

We'd be remiss not to briefly discuss Mixture of Experts (MoE) models^[4], which replace the single dense MLP blocks in a standard Transformer with a set of independent MLPs that can be dynamically routed between. To a first approximation, **an MoE is just a normal dense model with E MLP blocks per layer**, instead of just one. Each token activates k of these experts, typically $k \ll E$. The ratio E/k is called the sparsity and is usually between 8 and 64 (e.g. [DeepSeek v3](#) has effectively $k = 8$, $E = 256$). This increases the parameter count by $O(E)$, while multiplying the total number of activated parameters per token by k , compared with the dense version.

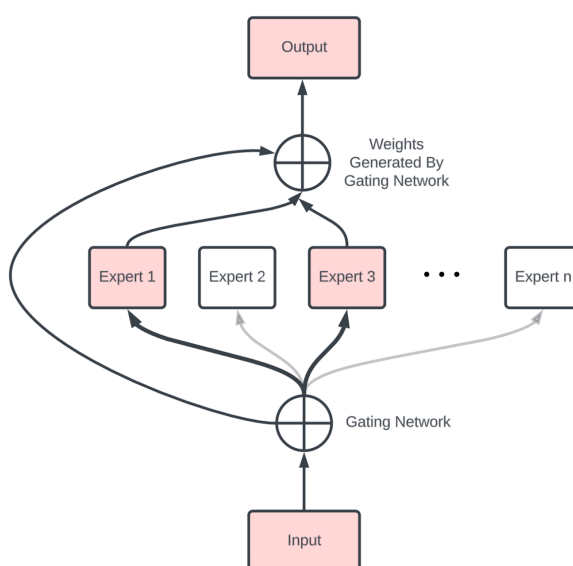


Figure: an example MoE layer with n experts. The gating expert routes each token to k of them, and the output of those k MLPs get summed. Our parameter count is n times the size of each expert, but only k are used for each token. [Source](#).

Compared to a dense model, an MoE introduces new comms, primarily two AllToAlls (one before and one after the MoE block) that route tokens to the correct expert and bring them back to their home device.³ However, as we saw in the previous section, the cost of each AllToAll is only 1/4 that of a comparable AllGather along a single axis (for a bidirectional ring).

Gradient checkpointing

Backpropagation as an algorithm trades compute for memory. Instead of a backward pass requiring $O(n_{\text{layers}}^2)$ FLOPs, it requires $O(n_{\text{layers}})$ memory, saving all intermediate activations generated during the forward pass. While this is better than quadratic compute, it's incredibly expensive memory-wise: a model with $B * T = 4M$ (4M total tokens per batch), L=64, and D=8192 that avoids all unnecessary backward pass compute would have to save roughly $2 * 20 * B * T * D * L = 84TB$ of activations in bfloat16. The 20 comes from (roughly) counting every intermediate node in the Transformer diagram above, since e.g.

$$f(x) = \exp(g(x))$$
$$\frac{df}{dx} = \exp(g(x)) \cdot \frac{dg}{dx}$$

so to avoid recomputing we need to save $g(x)$ and $\exp(g(x))$ from the forward pass. To avoid saving this much memory, we can choose to only save some fraction of the intermediate activations. Here are a few strategies we use.

- **Block remat**: only save the input to each layer. This is the most aggressive method we use and only saves 1 checkpoint per layer, meaning we'd only save 4.2TB in the example above. This forces us to repeat essentially all forward pass FLOPs in the backward pass, meaning we increase our FLOPs from $6 \cdot \text{num params} \cdot \text{num tokens}$ to roughly $8 \cdot \text{num params} \cdot \text{num tokens}$.
- **Big matmuls only**: another simple policy is to only save the outputs of large matmuls. This lets us avoid recomputing any large matmuls during the backward pass, but still makes us recompute other activation functions and parts of attention. This reduces the 20 per layer above to closer to 7 per layer.

This is by no means comprehensive. When using JAX, these are typically controlled by `jax.remat` / `jax.checkpoint` (you can read more [here](#)).

Key-Value (KV) caching

As we'll see in [Section 7](#), LLM inference has two key parts, prefill and generation.

- **Prefill** processes a long prompt and saves its attention activations in a Key-Value Cache (KV Cache) for use in generation, specifically the key-value projections in the attention block.
- **Generation** batches several of these KV caches together and samples tokens from each of them.

Each KV cache is then effectively an array of size $[2, S, L, K, H]$ where the 2 accounts for the keys and values. This is quite large! The total size of the Key-Value cache in int8 is $2SLKH$. For a moderately sized model with 8k context length, 64 layers, and $KH = NH = D = 8192$, this is $2 \cdot 8192 \cdot 64 \cdot 8192 = 8 \text{ ГнБ}$. You can see why we would want to use GMQA with $K \ll N$.

What Should You Take Away from this Section?

- The overall parameters and FLOPs of a Transformer are fairly easy to calculate, and are summarized here, assuming MHA (with batch size B, vocab size V, a sequence of length T, $D=d_{\text{model}}$, and $F=d_{\text{ff}}$):

Component	Params per layer	Training FLOPs per layer
MLP	3DF	18BTDF
Attention	4DNH	24BTDNH + 12BT ² NH
Other	2D	BTD
Vocab	DV (total, not per-layer)	12BTDV

- The parameter count of the MLP block dominates the total parameter count and the MLP block also dominates the FLOPs budget as long as the sequence length $T < 8D$.
- The total FLOPs budget during training is well approximated by $6 \cdot \text{num_params} \cdot \text{num_tokens}$ for reasonable context lengths.
- During inference, our KV caches are roughly $2 \cdot S \cdot L \cdot K \cdot H$ per cache (where K is the number of KV heads), although architectural modifications can often reduce this.

A Few Problems to Work

Question 1: How many parameters does a model with $D = 4096$, $F = 4 \cdot D$, $V = 32000$, and $L = 64$ have? What fraction of these are attention parameters? How large are our KV caches per token? *You can assume $N \cdot H = D$ and multi-head attention with int8 KVs.*

► [Click here for the answer.](#)

Question 2: How many total FLOPs are required to perform $A[B_x, D_y] *_{\text{D}} W[D_y, F]$ on $\{ 'X': 4, 'Y': 8, 'Z': 4 \}$? How many FLOPs are performed by each TPU?

► [Click here for the answer.](#)

Question 3: How many FLOPs are involved in performing $A[I, J, K, L] * B[I, J, M, N, O] \rightarrow C[K, L, M, N, O]$?

► [Click here for the answer.](#)

Question 4: What is the arithmetic intensity of grouped multi-query attention (ignoring the Q/K/V/O projections)? Give the answer as a function of the Q and KV sequence lengths T and S and the multi-query factor G . At what context length is attention FLOPs-bound? Given the HBM bandwidth of our TPUs, plot the effective relative cost of attention to the FFW block as the context length grows. Hint: assume we're using an efficient attention implementation that doesn't do any unnecessary reads/writes. Consider both the limits where $T = S$ and $T \ll S$.

► [Click here for the answer.](#)

Question 5: At what sequence length are self-attention FLOPs equal to the QKVO projection FLOPs?

► [Click here for the answer.](#)

Question 6: Say we only save the output of each of the 7 main matmuls in a Transformer layer during our forward pass (Q, K, V, O + the three FFW matrices). How many extra FLOPs do we need to "rematerialize" during the backward pass?

► [Click here for the answer.](#)

Question 7: DeepSeek v3 says it was trained for 2.79M H800 hours on 14.8T tokens ([source](#)). Given that it has 37B activated parameters, roughly what hardware utilization did they achieve? Hint: note that they used FP8 FLOPs without structured sparsity.

► [Click here for the answer.](#)

Question 8: Mixture of Experts (MoE) models have E copies of a standard dense MLP block, and each token activates k of these experts. What batch size in tokens is required to be compute-bound for an MoE with weights in int8 on TPU v5e? For DeepSeek, which has 256 (routed) experts and $k = 8$, what is this number?

► [Click here for the answer.](#)

That's it for Part 4! For Part 5 (about scaling Transformer training), [click here!](#)

Appendix

Appendix A: How does Flash Attention work?

The traditional objection to scaling Transformers to very long context is that the attention FLOPs and memory usage scale quadratically with context length. While it's true that the attention QK product has shape $[B, T, S, N]$ where B is the batch size, T and S are the Q and K sequence dims, and N is the number of heads, this claim comes with some serious caveats:

1. As we noted earlier, even though this is quadratic, the attention FLOPs only dominate when $T > 8 \cdot D$, and during training the memory of a single attention matrix is small compared to all of the weights and activation checkpoints living in memory, especially when sharded.
2. We don't need to materialize the full attention matrix in order to compute attention! We can compute local sums and maxes and avoid ever materializing more than a small chunk of the array. While the total FLOPs is still quadratic, we drastically reduce memory pressure.

This second observation was first made by [Rabe et al. 2021](#) and later in the [Flash Attention paper](#) (Dao et al. 2022). The basic idea is to compute the attention in chunks of K/V, where we compute the local softmax and some auxiliary statistics, then pass them on to the next chunk which combines them with its local chunk. Specifically, we compute

1. **M**: The running max of $q \cdot k$ over the sequence dimension
2. **O**: The running full attention softmax over the sequence dimension
3. **L**: The running denominator $\sum_i \exp(q \cdot k_i - \text{running max})$

With these, we can compute the new max, the new running sum, and the new output with only a constant amount of memory. To give a sketchy description of how this works, attention is roughly this operation:

$$\text{Attn}(Q, K, V) = \sum_i \frac{\exp(Q \cdot K_i - \max_j Q \cdot K_j) V_i}{\sum_l \exp(Q \cdot K_l - \max_j Q \cdot K_j)}$$

The max is subtracted for numerical stability and can be subtracted without affecting the outcome since $\sum_i \exp(a_i + b) = \exp(b) \sum \exp(a)$. Looking just at the denominator above, if we imagine having two contiguous chunks of key vectors, K^1 and K^2 and we compute the local softmax sums L^1 and L^2 for each

$$L^1 = \sum_i \exp(Q \cdot K_i^1 - \max_j Q \cdot K_j^1)$$

$$L^2 = \sum_i \exp(Q \cdot K_i^2 - \max_j Q \cdot K_j^2)$$

Then we can combine these into the full softmax sum for these two chunks together by using

$$L^{\text{combined}} = \exp(M^1 - \max(M^1, M^2)) \cdot L^1 + \exp(M^2 - \max(M^1, M^2)) \cdot L^2$$

where

$$M^1 = \max_j Q \cdot K_j^1 \text{ and } M^2 = \max_j Q \cdot K_j^2$$

This can be done for the full softmax as well, giving us a way of accumulating arbitrarily large softmax sums. Here's the full algorithm from the Flash Attention paper.

Algorithm 1 FLASHATTENTION

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M .

- 1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil, B_r = \min(\lceil \frac{M}{4d} \rceil, d)$.
 - 2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
 - 3: Divide $\mathbf{Q}$ into $T_r = \lceil \frac{N}{B_r} \rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ into $T_c = \lceil \frac{N}{B_c} \rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
 - 4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
 - 5: **for** $1 \leq j \leq T_c$ **do**
 - 6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
 - 7: **for** $1 \leq i \leq T_r$ **do**
 - 8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
 - 9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
 - 10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}, \tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
 - 11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}, \ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
 - 12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
 - 13: Write $\ell_i \leftarrow \ell_i^{\text{new}}, m_i \leftarrow m_i^{\text{new}}$ to HBM.
 - 14: **end for**
 - 15: **end for**
 - 16: Return $\mathbf{O}$.
-

From a hardware standpoint, this lets us fit our chunk of $\mathbf{Q}$ into VMEM (what the algorithm above calls on-chip SRAM) so we only have to load the $\mathbf{KV}$ chunks on each iteration, increasing the arithmetic intensity. We can also keep the running statistics in VMEM.

One last subtle point worth emphasizing is an attention softmax property that's used to make the Flash VJP (reverse mode derivative) calculation practical for training. We define an intermediate softmax array:

$$S_{ij} = \frac{e^{\tau q_i \cdot k_j}}{\sum_l e^{\tau q_i \cdot k_l}}$$

In attention, we obtain dS from reverse-mode dO and V arrays:

$$dS_{ij} = dO_{id} \cdot_d V_{jd} = \sum_d dO_{id} V_{jd}$$

During the backpropagation of this gradient to $\mathbf{Q}$ and $\mathbf{K}$

$$d(q_i \cdot k_j) = (dS_{ij} - S_{ij} \cdot_j dS_{ij}) S_{ij}$$

We exploit an identity that allows us to exchange a contraction along the large key **length** dimension with a local contraction along the feature **depth** dimension.

$$\begin{aligned}
S_{ij} \cdot_j dS_{ij} &= \sum_j \frac{e^{\tau q_i \cdot k_j}}{\sum_k e^{\tau q_i \cdot k_k}} \sum_d dO_{id} V_{jd} \\
&= \sum_d dO_{id} \sum_j \frac{e^{\tau q_i \cdot k_j}}{\sum_k e^{\tau q_i \cdot k_k}} V_{jd} \\
&= \sum_d dO_{id} O_{id} \\
&= dO_{id} \cdot_d O_{id}
\end{aligned}$$

This replacement is crucial for being able to implement a sequence-block *lca*/calculation for the VJP, and enables further clever sharding schemes like ring attention.

Footnotes

1. **Сжимаемые** измерения — это оси, которые суммируются в ходе операции (они присутствуют на обоих входах, но отсутствуют на выходе), как внутреннее измерение при умножении матриц. **Объединяемые** измерения — это общие оси, которые присутствуют на обоих входах и без изменений передаются на выход; они индексируют независимые подзадачи и не умножаются друг на друга при подсчете количества операций с плавающей запятой. В терминах einsum: метки, присутствующие на обоих входах и на выходе, объединяются; метки, присутствующие на обоих входах, но отсутствующие на выходе, сжимаются. [\[↗\]](#)
2. Обратите внимание, что в некоторых современных моделях OSS используется локальное внимание или другие методы оптимизации, которые снижают затраты на внимание и меняют эту закономерность. [\[↗\]](#)
3. Технически это происходит только в том случае, если наши данные или последовательности сегментированы по той же оси, что и наши эксперты. [\[↗\]](#)

References

1. **GLU Variants Improve Transformer**
Shazeer, N., 2020. arXiv [cs.LG].
2. **Fast Transformer decoding: One write-head is all you need**
Shazeer, N., 2019. arXiv [cs.NE].
3. **GQA: Training generalized multi-query transformer models from multi-head checkpoints**
Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F. and Sanghai, S., 2023. arXiv [cs.CL].
4. **Outrageously large neural networks: The Sparsely-Gated Mixture-of-experts layer**
Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G. and Dean, J., 2017. arXiv [cs.LG].

Miscellaneous

*Work done at Google DeepMind, now at MatX.

Citation

For attribution in academic contexts, please cite this work as:

Austin et al., "How to Scale Your Model", Google DeepMind, online, 2025.

or as a BibTeX entry:

```
@article{scaling-book,
  title = {How to Scale Your Model},
  author = {Austin, Jacob and Douglas, Sholto and Frostig, Roy and Levskaya, Anselm and Chen, Charlie and Vikram, Sharad and Lebron, Federico and Choy, Peter and Ramasesh, Vinay and Webson, Albert and Pope, Reiner},
  publisher = {Google DeepMind},
  howpublished = {Online},
  note = {Retrieved from https://jax-ml.github.io/scaling-book/},
  year = {2025}
}
```